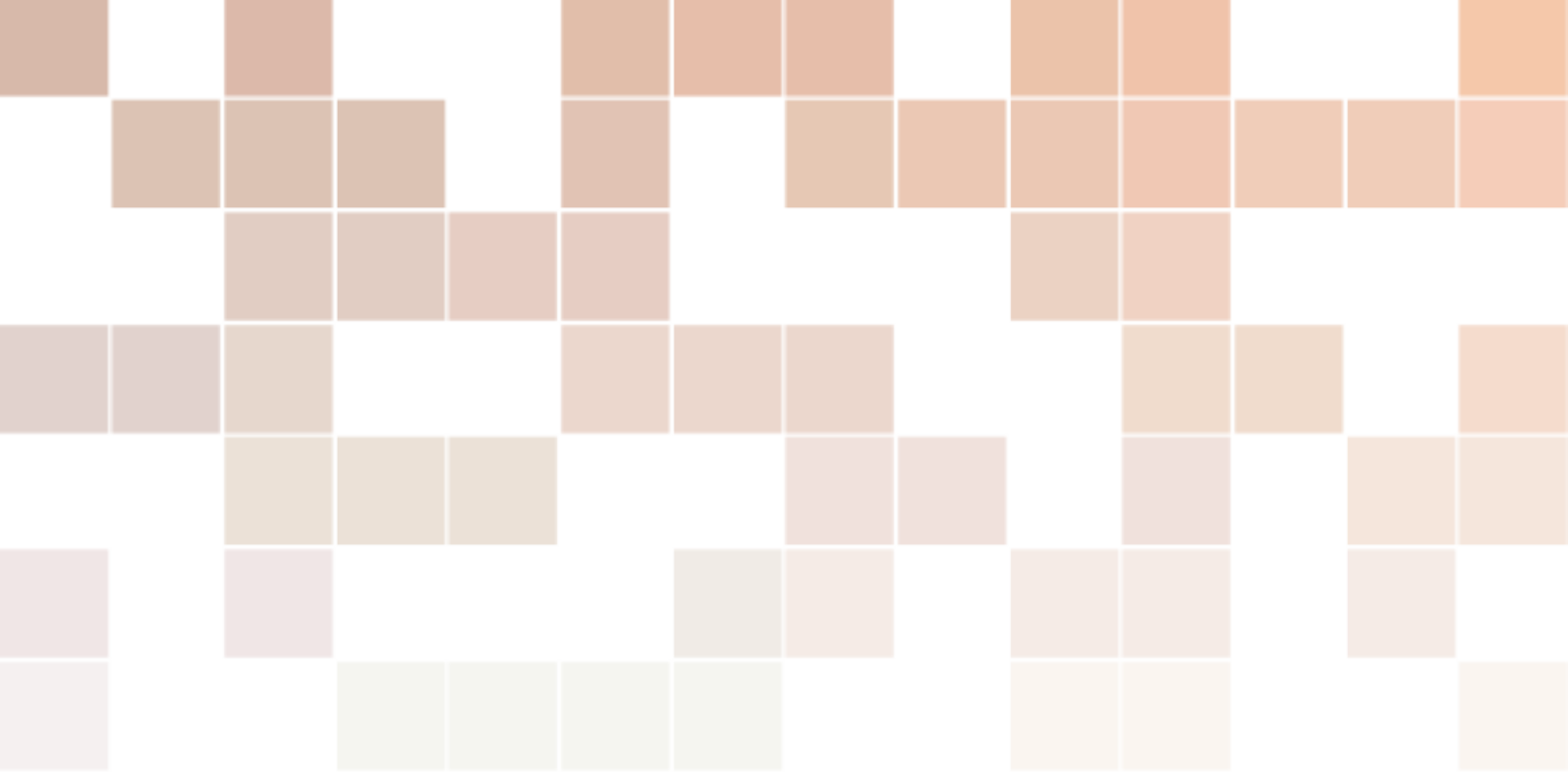
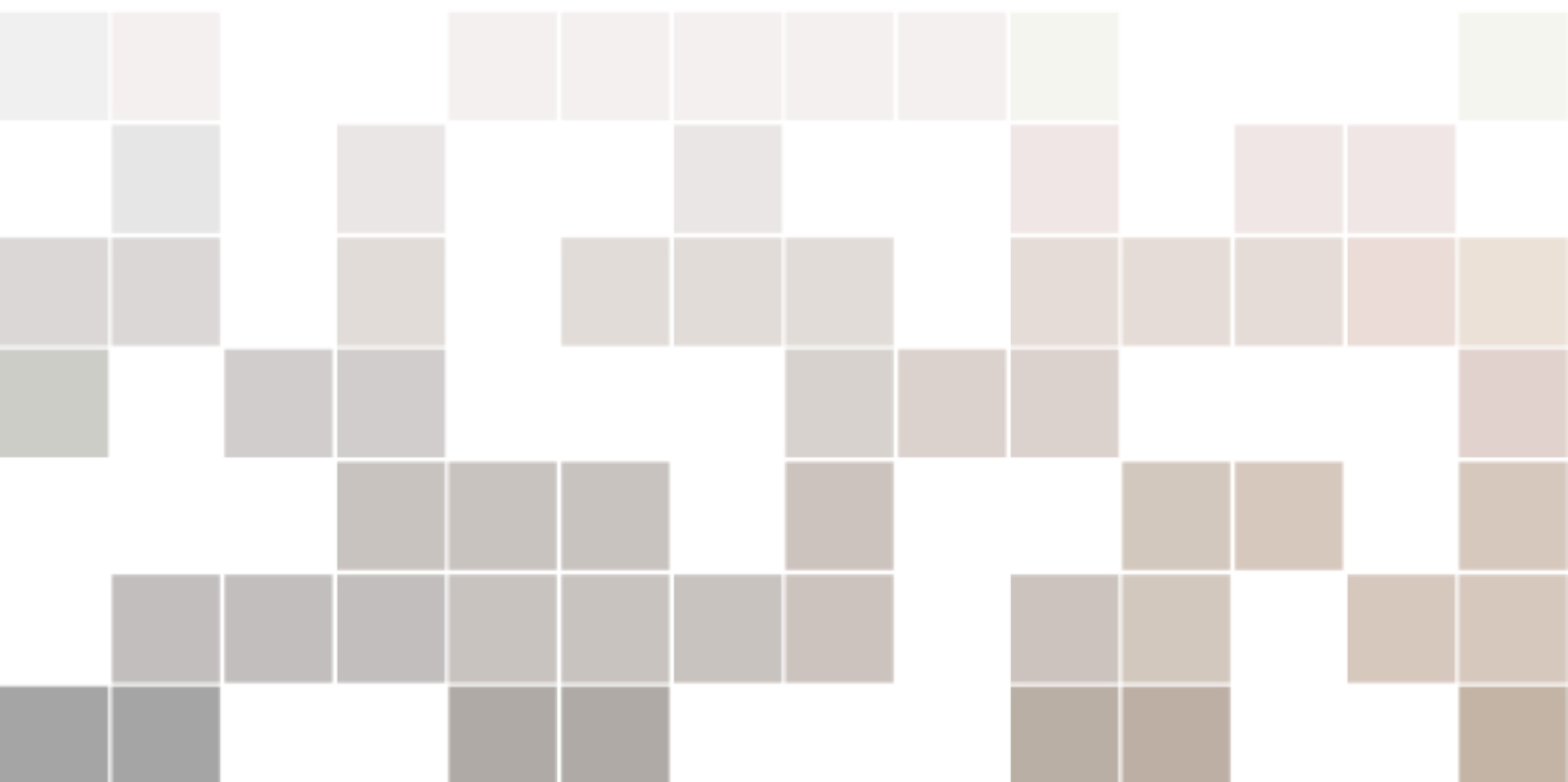




YAQT

Yet Another Quantum Theory

Justin Kelly





Contents

	Preface	5
	The Pixel and the Maxel	5
	Advanced Operations	5
1	Idris2 Base	7
1.1	About	7
1.1.1	Types	7
1.1.2	Singletons	7
1.1.3	Pixels	7
1.1.4	Maxel Arithmetic	8
1.1.5	Vexel Dot Products	8
1.1.6	Diamond Poset Transitivity	8
1.1.7	Executing the Tests	8
1.2	Properties	8
1.2.1	Ordered Structures	8
1.2.2	Graphs	9
1.2.3	Equivalence Relations	10
1.2.4	Relations	11
1.2.5	Tournaments	11
	Bibliography	13

Preface

i Chapter Summary

Chapter Intent: This chapter describes Singletons (`Sing`), Pixels (`Pix`) and Maxels (`Maxel`)- foundational types in Wildbergers Box Arithmetic, these are defined as finite collections of Multisets (`LMset`).

The primary focus of this library is verifying structural mathematical properties (e.g., reflexivity, transitivity, symmetry) of these objects using the dependently typed programming language Idris 2 [1].

Welcome to the **Base module**! This chapter introduces the foundational data structures for the RCIT framework.

As discussed by Wildberger [2], we build upon the `LMset` structures to define our metric spaces. Let us look at the core imports and our first type, `Sing`.

i Note

Notice how we use dependent types to guarantee that every `Sing` contains exactly one `LMset`.

The Pixel and the Maxel

A `Pix` (Pixel) represents a directed edge between two `LMset` nodes. A `Maxel` is an aggregation of multiple pixels, which can be thought of mathematically as an adjacency matrix or a multigraph.

Mathematically, if a pixel p connects node a to node b , its matrix representation would be:

$$P_{ij} = \begin{cases} 1 & \text{if } i = a \text{ and } j = b \\ 0 & \text{otherwise} \end{cases} \quad (0.1)$$

Advanced Operations

When multiplying pixels together, we must ensure the destination node of the first pixel matches the source node of the second pixel. This is formally verified in Idris [1].

Here we instantiate these concepts to test their behavior.

1. Idris2 Base

Singletons are single mutisets. Pixels are pairs of singletons. Maxels are multisets of pixels.

1.1 About

1.1.1 Types

This section demonstrates the basic instantiation and arithmetic of foundational data structures in the `Base` module. These tests verify the correct behavior of singletons, pixels, maxels, and their operations before moving on to more complex property-based testing.

```
module Main

import Base
import LMset
```

1.1.2 Singletons

First, we will instantiate a singleton (`Sing`).

```
testSingletons : IO ()
testSingletons = do
  let s = MkSing (fromNat 5)
  putStrLn $ "Singleton: " ++ show s
```

1.1.3 Pixels

We now create directed edges (`Pix`) between basic numbers. A `Pix` represents a directed connection from a source node to a target node.

```
-- Create pixels (representing directed edges between numbers)
let p1 = MkPix (fromNat 1) (fromNat 2) -- 1 -> 2
let p2 = MkPix (fromNat 2) (fromNat 3) -- 2 -> 3

putStrLn $ "p1: " ++ show p1
putStrLn $ "p2: " ++ show p2
```

1.1.4 Maxel Arithmetic

A `Maxel` aggregates multiple pixels. Here we verify both addition (which merges the subsets of pixels) and multiplication (which composes compatible edges). When we multiply maxels, a new pixel is formed only when the destination of an edge in the first maxel matches the source of an edge in the second.

```
testMaxelArithmetic : IO ()
testMaxelArithmetic = do
  let p1 = MkPix (fromNat 1) (fromNat 2)
  let p2 = MkPix (fromNat 2) (fromNat 3)
  let p3 = MkPix (fromNat 3) (fromNat 4)

  -- Create sparse maxels
  let m1 = MkMaxel [p1]
  let m2 = MkMaxel [p2, p3]

  putStrLn $ "m1 + m2: " ++ show (addMaxel m1 m2)
  putStrLn $ "m1 * m2: " ++ show (mulMaxel m1 m2)
```

1.1.5 Vexel Dot Products

We can simulate vector-like structures using “vexels” (vectors of pixels). A row vexel and a column vexel can be multiplied together to compute their dot product.

```
testVexelDotProduct : IO ()
testVexelDotProduct = do
  let vRow = rowVexel (fromNat 1) [fromNat 2, fromNat 3]
  let vCol = colVexel (fromNat 4) [fromNat 2, fromNat 3]

  putStrLn $ "Row Vexel: " ++ show vRow
  putStrLn $ "Col Vexel: " ++ show vCol
  putStrLn $ "Vexel Dot Product (vRow * vCol): " ++ show (mulMaxel vRow vCol)
```

1.1.6 Diamond Poset Transitivity

Finally, we test the hardcoded `diamondMaxel`. By squaring it (multiplying it by itself), we verify that its structure correctly models mathematical transitivity.

```
testDiamondPoset : IO ()
testDiamondPoset = do
  putStrLn $ "Diamond Maxel: " ++ show diamondMaxel
  putStrLn $ "Diamond Maxel Squared (Transitivity): " ++ show (mulMaxel diamondMaxel
    diamondMaxel)
```

1.1.7 Executing the Tests

With all our test definitions in place, we can execute them sequentially in our `main` function.

```
main : IO ()
main = do
  testSingletons
  testMaxelArithmetic
  testVexelDotProduct
  testDiamondPoset
```

1.2 Properties

1.2.1 Ordered Structures

This module covers combinatorics.


```

module Main

import Hedgehog
import LMset
import Base
import Combinatorics

genNode : Gen LMset
genNode = do
  n <- nat (linear 1 3)
  pure $ fromNat n

genPix : Gen Pix
genPix = do
  a <- genNode
  b <- genNode
  pure $ MkPix a b

genMaxel : Gen Maxel
genMaxel = do
  -- Small size to naturally hit property intersections
  pixels <- list (linear 0 4) genPix
  pure $ MkMaxel pixels

prop_T_plus_Y_implies_R : Property
prop_T_plus_Y_implies_R = property $ do
  m <- forAll genMaxel
  if isTransitive m && isSymmetric m
    then isReflexive m == True
    else assert True

genPoset : Gen Maxel
genPoset = pure diamondMaxel

prop_poset_properties : Property
prop_poset_properties = property $ do
  -- diamondMaxel is a strict poset
  m <- forAll genPoset
  isTransitive m == True
  isAntiSymmetric m == True
  isIrreflexive m == True

main : IO ()
main = do
  success <- checkGroup $ MkGroup "Ordered Structures"
    [ ("prop_T_plus_Y_implies_R", prop_T_plus_Y_implies_R)
    , ("prop_poset_properties", prop_poset_properties)
    ]
  if success
    then putStrLn "SUCCESS"
    else putStrLn "FAILURE"

```

1.2.2 Graphs

```

module Main
import Hedgehog
import LMset
import Base
import Combinatorics

```

```

genNode : Gen LMset
genNode = do
  n <- nat (linear 1 3)
  pure $ fromNat n

genPix : Gen Pix
genPix = do
  a <- genNode
  b <- genNode
  pure $ MkPix a b

genMaxel : Gen Maxel
genMaxel = do
  pixels <- list (linear 0 4) genPix
  pure $ MkMaxel pixels

prop_undirected_symmetric : Property
prop_undirected_symmetric = property $ do
  m <- forAll genMaxel
  -- M + M^T is always symmetric!
  let undirected = addMaxel m (transposeMaxel m)
  isSymmetric undirected === True

main : IO ()
main = do
  success <- checkGroup $ MkGroup "Graphs"
  [ ("prop_undirected_symmetric", prop_undirected_symmetric) ]
  if success then putStrLn "SUCCESS" else putStrLn "FAILURE"

```

1.2.3 Equivalence Relations

```

module Main
import Hedgehog
import LMset
import Base
import Combinatorics

genNode : Gen LMset
genNode = do
  n <- nat (linear 1 3)
  pure $ fromNat n

-- A complete graph (clique) over a set of nodes is always an equivalence relation
genClique : Gen Maxel
genClique = do
  nodes <- list (linear 1 3) genNode
  -- Generate every possible a->b pair
  pure $ MkMaxel [ MkPix a b | a <- nodes, b <- nodes ]

prop_clique_is_equivalence : Property
prop_clique_is_equivalence = property $ do
  m <- forAll genClique
  isReflexive m === True
  isSymmetric m === True
  isTransitive m === True

main : IO ()
main = do

```

```

success <- checkGroup $ MkGroup "Equivalence Relations"
[ ("prop_clique_is_equivalence", prop_clique_is_equivalence) ]
if success then putStrLn "SUCCESS" else putStrLn "FAILURE"

```

1.2.4 Relations

```

module Main
import Hedgehog
import LMset
import Base
import Combinatorics

genNode : Gen LMset
genNode = do
  n <- nat (linear 1 3)
  pure $ fromNat n

genPix : Gen Pix
genPix = do
  a <- genNode
  b <- genNode
  pure $ MkPix a b

genMaxel : Gen Maxel
genMaxel = do
  pixels <- list (linear 0 4) genPix
  pure $ MkMaxel pixels

prop_transpose_involution : Property
prop_transpose_involution = property $ do
  m <- forAll genMaxel
  -- Any Maxel is a relation (directed multigraph)
  -- The transpose of the transpose is the original maxel!
  transposeMaxel (transposeMaxel m) == m

main : IO ()
main = do
  success <- checkGroup $ MkGroup "Relations"
  [ ("prop_transpose_involution", prop_transpose_involution) ]
  if success then putStrLn "SUCCESS" else putStrLn "FAILURE"

```

1.2.5 Tournaments

```

module Main
import Hedgehog
import LMset
import Base
import Combinatorics

genTournament : Gen Maxel
genTournament = do
  -- A simple hardcoded tournament on 3 nodes: 1->2, 2->3, 1->3
  pure $ MkMaxel [
    MkPix (fromNat 1) (fromNat 2),
    MkPix (fromNat 2) (fromNat 3),
    MkPix (fromNat 1) (fromNat 3)
  ]

```

```
prop_is_tournament : Property
prop_is_tournament = property $ do
  m <- forAll genTournament
  isSet m == True
  isIrreflexive m == True
  isAntiSymmetric m == True
  isTotal m == True

main : IO ()
main = do
  success <- checkGroup $ MkGroup "Tournaments"
    [ ("prop_is_tournament", prop_is_tournament) ]
  if success then putStrLn "SUCCESS" else putStrLn "FAILURE"
```



Bibliography

- [1] E. Brady, *Type-Driven Development with Idris*. Manning Publications, 2017.
- [2] N. Wildberger, “Box Arithmetic and LMset,” *Journal of Chromogeometry*, vol. 14, pp. 22–45, 2020.